

Quiz-App

Complete Reference Document

A comprehensive guide to the features, design system, and technical architecture of the Quiz-App platform. Written as a framework-agnostic reference for rebuilding from scratch with any technology stack.

League of Amazing Programmers

July 2026

Table of Contents

1. What This App Does

- 1.1 The Five Modes
- 1.2 Feature Inventory by Mode
- 1.3 Shared Systems (Auth, Gamification, AI)

2. Design System

- 2.1 Color Palette
- 2.2 Typography
- 2.3 Spacing Grid
- 2.4 Elevation & Shadows
- 2.5 Border Radius Tokens
- 2.6 Component Patterns
- 2.7 Animation & Motion
- 2.8 Responsive Strategy
- 2.9 Icon System

3. Technical Architecture

- 3.1 System Overview
- 3.2 Authentication Flow
- 3.3 API Endpoints
- 3.4 Data Models
- 3.5 AI Integration
- 3.6 Content System
- 3.7 Performance Patterns
- 3.8 Environment & Configuration

4. Jargon Glossary

1. What This App Does

Quiz-App is a gamified learning platform built for high school students at the League of Amazing Programmers. It teaches students how to use AI as a tool in the software development lifecycle (SDLC) -- planning, designing, and building real applications -- rather than using AI to do the work for them. The platform has five distinct modes, each serving a specific pedagogical purpose.

SDLC (Software Development Lifecycle) -- The process of planning, creating, testing, and deploying software. In this app, the five phases are Discover, Analyze, Design, Build, and Ship.

Gamification -- Using game-like elements (points, levels, streaks, leaderboards) in a non-game context to motivate engagement.

1.1 The Five Modes

Mode	Route	Purpose	Status
Dashboard	/dashboard	Central hub showing progress, skills, and navigation to other modes	Complete
Learn	/learn	Course catalog for structured lessons (video, reading, interactive)	Placeholder
Crucible	/crucible	Assessment hub -- take quizzes, review history, compete on leaderboards	Complete
Workspaces	/workspaces	Browser-based IDE for coding projects	Placeholder
Studio	/course/build-real-stuff	AI-assisted project builder (Codecademy-inspired)	Prototype

Placeholder -- A "coming soon" page with a description of planned features. The UI shell exists but there is no functional backend.

1.2 Feature Inventory by Mode

Dashboard

The landing page after login. Shows the student's GitHub avatar, username, role badge, and three stat cards (Total Points, Quizzes Taken, Global Rank). Below that, animated skill bars break down performance by category (Python, Java, Technology). Three "zone cards" link to Learn, Crucible, and Studio with descriptive summaries and activity counts.

- Profile hero card with GitHub avatar (pulled from github.com/{username}.png)
- 3-column stats grid: Total Points, Quizzes Taken, Global Rank
- Animated skill bars colored by category, showing percentage distribution
- Three zone cards linking to Learn, Crucible, and Studio
- Skeleton loading UI during authentication transition
- Themed animated canvas background (floating achievement symbols)

Crucible (Assessment Hub)

The quiz-taking and review center. Three tabs: Arena for browsing and taking quizzes by category, Gauntlet for reviewing past attempts, and Leaderboard for competitive ranking.

- **Arena tab:** Category cards (Python, Java, Technology) with large SVG icons. Clicking a category shows filtered quiz cards with difficulty badges, question counts, completion status, and best scores.
- **Gauntlet tab:** Expandable history groups per quiz showing every past attempt with timestamps and scores. "Rematch" button for retakes.
- **Leaderboard tab:** Top-10 global ranking with medal emojis for top 3.
- Seven question types: Multiple Choice, Parsons Problems (drag-and-drop code ordering), Faded Parsons (partial ordering with fixed hints), Output Prediction, Debugging, Fill in the Blank, Free Response.
- Retake scoring: only newly-correct answers earn points on repeat attempts.

***Parsons Problem** -- A question type where code lines are presented out of order and the student must drag them into the correct sequence. Tests code comprehension without requiring the student to write code from scratch.*

***Faded Parsons** -- A scaffolded variant where some lines are fixed in place (shown grayed out) and the student only reorders the remaining lines. Reduces cognitive load.*

Studio (AI Builder)

A Codecademy-inspired AI builder where students describe what they want to build and LeagueAI helps them plan, design, and build it step by step. Two states:

- **Welcome state:** Centered prompt ("What do you want to build?") with suggestion chips (habit tracker, recipe finder, portfolio, game, help finding an idea) and a chat input.
- **Workspace state:** Split view with chat panel (left) and live preview (right). Build/Learn tab toggle. Build tab shows chat + preview; Learn tab shows milestone-based learning content (4 milestones with task checkpoints).
- Subtle SDLC phase indicator at bottom (Discover, Analyze, Design, Build, Ship as pills).
- Rank badge (Explorer through Shipper based on XP) and token counter in header.
- Code view toggle on preview panel to see generated code.
- **Note:** AI backend is not yet connected -- chat returns a canned placeholder response.

Quest System (within Studio)

The Studio course has individual quests -- structured activities that go beyond quizzes. Two quest types currently exist:

- **Reflection Journal:** A textarea with word-count progress bar. Students write reflections that must meet a minimum word count. Drafts auto-save to browser storage.
- **AI Chat Challenge:** Embeds the AI Tutor chat widget. Students converse with the AI tutor (Socratic style), then submit the conversation for rubric-based grading. Each quest has a credit budget limiting AI usage.
- Each quest awards 10 XP on completion. Streaks track consecutive daily activity.

***Rubric-based grading** -- The AI evaluates the student's conversation against specific criteria defined in the quest's YAML file. Each criterion produces a pass/fail with evidence text.*

Login Page

Full-screen space-themed experience with an elaborate canvas animation (3D stars, planets, floating code snippets, shooting stars, nebulae). Single login method: GitHub OAuth. Multi-step cinematic transition on login: card morphs to loading bar, warp-speed fade effect, then dashboard appears with skeleton loading.

1.3 Shared Systems

Authentication

GitHub OAuth is the only authentication method. After the OAuth redirect, the backend stores the user's GitHub ID and username in a signed session cookie. There are no passwords, no JWT tokens, and no database-backed sessions. Three roles exist: Student (default), Teacher, and Developer.

***OAuth** -- A protocol that lets users log in via a third-party service (GitHub) without sharing their password. The app redirects to GitHub, the user approves, GitHub sends back a code, and the app exchanges that code for user information.*

***Session cookie** -- A small piece of data stored in the browser that identifies the user on each request. Signed (cryptographically tamper-proof) so it cannot be forged.*

Gamification

- Points: earned by correctly answering quiz questions (retakes only award points for new correct answers)
- Ranks: Explorer (Lv 1) through Shipper (Lv 10+), calculated as $\text{floor}(\text{XP} / 50) + 1$
- Streaks: increment when last activity was within 24 hours, reset to 1 otherwise
- Leaderboard: top 10 users by total points, with medal badges for top 3
- Click particles: subtle colored particles burst from clicked interactive elements

AI Integration

The AI tutor uses OpenRouter (an API aggregator) calling DeepSeek models. Two tiers: "fast" (deepseek-v4-flash) for chat and grading, "agent" (deepseek-v4-pro) reserved for future features. Each quest has a credit budget measured in tokens -- when exhausted, the student can no longer chat with the AI for that quest. Rate limiting prevents abuse (6 calls per 60 seconds per user).

***OpenRouter** -- A service that provides a unified API to access many different AI models (OpenAI, Anthropic, DeepSeek, etc.) through a single endpoint.*

***Token** -- A unit of text processed by an AI model. Roughly 4 characters = 1 token. Credit budgets limit total tokens spent per quest.*

Animated Backgrounds

A reusable canvas-based component supports 14 named themes (e.g., neural, geometric, ocean, matrix, circuit, dashboard). Each theme draws different animated elements. Frame-rate capped at 30fps. Respects the prefers-reduced-motion accessibility setting.

2. Design System

The visual language is defined through CSS custom properties (variables) in a central file. All components reference these tokens rather than hard-coding values, making the theme easy to change. The aesthetic is clean, light, and modern with a unified orange/navy palette -- no per-section color overrides.

CSS Custom Properties (variables) -- Named values like `--color-primary: #ef6c00` defined once and reused everywhere. Changing the value in one place updates the entire app.

Design Tokens -- The smallest pieces of a design system -- colors, spacing values, font sizes, border radii, shadows. Named abstractly (e.g., `--space-4` rather than `16px`) so they can be adjusted without touching component code.

2.1 Color Palette

Brand Colors

Token	Value	Usage
<code>--color-primary</code>	<code>#ef6c00</code>	Main orange -- buttons, links, accents, focus rings
<code>--color-primary-dark</code>	<code>#e65100</code>	Hover/active state of primary elements
<code>--color-primary-light</code>	<code>#ff9800</code>	Gradient endpoints, lighter accents
<code>--color-primary-subtle</code>	<code>rgba(239,108,0,0.08)</code>	Tinted backgrounds (hover chips, badges)
<code>--color-primary-glow</code>	<code>rgba(239,108,0,0.15)</code>	Focus ring glow around inputs
<code>--color-secondary</code>	<code>#1a365d</code>	Navy -- headings, sidebar, secondary elements
<code>--color-secondary-dark</code>	<code>#0d1b2a</code>	Deepest navy for high contrast
<code>--color-secondary-light</code>	<code>#2d4a7a</code>	Lighter navy for hover states

Semantic Colors

Token	Value	Light Variant
<code>--color-success</code>	<code>#16a34a</code>	<code>--color-success-light: #dcfce7</code>
<code>--color-error</code>	<code>#dc2626</code>	<code>--color-error-light: #fee2e2</code>
<code>--color-warning</code>	<code>#d97706</code>	<code>--color-warning-light: #fef3c7</code>
<code>--color-info</code>	<code>#2563eb</code>	<code>--color-info-light: #dbeafe</code>

Surface & Text Colors

Token	Value	Usage
<code>--bg-app</code>	<code>#f8fafc</code>	Page background
<code>--bg-card</code>	<code>#ffffff</code>	Card backgrounds

Token	Value	Usage
--bg-muted	#f1f5f9	Muted/subtle backgrounds
--bg-dark	#0f172a	Dark surfaces (code blocks)
--text-primary	#0f172a	Headings, body text
--text-secondary	#475569	Descriptions, secondary labels
--text-muted	#94a3b8	Placeholder text, disabled states
--border-default	#e2e8f0	Card borders, dividers
--border-focus	#ef6c00	Focus ring color

Design decision: The color palette is heavily influenced by Tailwind CSS's Slate and Orange scales. This is intentional -- if you rebuild with Tailwind, the mapping is nearly 1:1 (e.g., --text-primary = slate-900, --bg-app = slate-50).

2.2 Typography

Role	Font	Weights	Usage
Headings	Poppins (Google Fonts)	400, 500, 600, 700	All h1-h4, nav brand, stat labels
Body	Open Sans (Google Fonts)	400, 500, 600, 700	Body text, buttons, inputs, descriptions
Code	Fira Code / Courier New	400	Code blocks, syntax highlighting, monospaced content

Type Scale

Token	Size	Typical Use
--text-xs	12px (0.75rem)	Badges, metadata, timestamps
--text-sm	14px (0.875rem)	Card descriptions, secondary text, buttons
--text-base	16px (1rem)	Body text, input text
--text-lg	18px (1.125rem)	H4 headings, section subheads
--text-xl	20px (1.25rem)	H3 headings
--text-2xl	24px (1.5rem)	H2 headings
--text-3xl	30px (1.875rem)	H1 headings (rare)
--text-4xl	36px (2.25rem)	Page titles, hero text

Key rules: All headings use Poppins at weight 600, line-height 1.3, letter-spacing -0.01em. Body text uses Open Sans at weight 400, line-height 1.6. Labels use uppercase + 0.05em letter-spacing + weight 600 + text-xs.

2.3 Spacing Grid

The spacing system uses a 4px base unit (8dp grid). Most values are multiples of 4px. The scale intentionally skips some values (no --space-7, --space-9, etc.).

Token	Value	Common Use
--space-1	4px	Tight gaps between icons and text
--space-2	8px	Gaps in button groups, chip rows
--space-3	12px	Internal padding of compact elements
--space-4	16px	Default gap in grids and lists
--space-5	20px	Card internal padding (compact)
--space-6	24px	Card internal padding (standard), section gaps
--space-8	32px	Section separation, hero padding
--space-10	40px	Large section spacing
--space-12	48px	Empty state padding, hero sections
--space-16	64px	Maximum spacing (rare)

2.4 Elevation & Shadows

Shadows are subtle (max 8% opacity). Cards rely on borders for definition at rest and gain shadows only on hover. This creates a clean, flat-ish aesthetic.

Token	Value	Usage
--shadow-sm	0 1px 2px rgba(0,0,0,0.05)	Resting cards (subtle lift)
--shadow-md	0 4px 6px rgba(0,0,0,0.07)	Hover state for cards and buttons
--shadow-lg	0 10px 15px rgba(0,0,0,0.08)	Elevated panels, skeleton loaders
--shadow-xl	0 20px 25px rgba(0,0,0,0.08)	Modals, overlays

2.5 Border Radius Tokens

Token	Value	Usage
--radius-sm	4px	Small elements, inner corners
--radius-md	8px	Buttons, inputs, tab groups
--radius-lg	12px	Cards, panels, chat bubbles
--radius-xl	16px	Large cards, modals, login container
--radius-full	9999px	Pill buttons, badges, avatars, inputs

Design decision: All primary/secondary/outline buttons use pill shape (radius-full). This is a distinctive part of the visual identity.

2.6 Component Patterns

These are the repeating visual patterns used across the app. When rebuilding, recreate these patterns to maintain the same feel.

Cards

- **Base card:** White bg, 12px radius, 24px padding, 1px border. Hover: translateY(-2px) + shadow-md.
- **Quiz card:** Base + 4px orange left border. Completed: border turns green.
- **Stat card:** Centered content, 4px orange top border.
- **Zone card:** Horizontal layout (icon + content + chevron). Each zone has a unique icon background color.
- **Category card:** Fixed 200px height, centered icon, hover lifts + orange border.

Badges

All badges use pill shape (border-radius: 9999px). Variants: rank badge (orange gradient), difficulty badge (color-coded by level), completion badge (green check), score badge, role badge, attempt count badge.

Chat Bubbles

- User messages: right-aligned, orange background, white text, rounded with one squared corner (bottom-right).
- AI messages: left-aligned, light gray background, dark text, rounded with one squared corner (bottom-left). AI avatar circle on the left.

Form Elements

- Inputs: 1px border, 8px radius, 16px body font. Focus: orange border + 3px orange glow ring.
- All buttons: pill shape for primary/secondary/outline variants.

2.7 Animation & Motion

Transition Tokens

Token	Value	Usage
--transition-fast	150ms ease	Color changes, border changes
--transition-base	200ms ease	Standard hover effects
--transition-slow	300ms ease	Complex transitions

Easing Philosophy

- **ease** -- simple transitions (hover, color)
- **cubic-bezier(0.16, 1, 0.3, 1)** -- entrance animations (overshoot/spring feel)
- **cubic-bezier(0.4, 0, 0.2, 1)** -- exit animations (Material Design standard)

Hover Pattern

Cards lift translateY(-2px) + shadow-md on hover. Buttons lift translateY(-1px). This is consistent across the entire app. Interactive elements also transition border-color to orange on hover.

Notable Animations

- **Login transition:** Multi-step cinematic -- card morphs to loading bar (0.4s), content fades (0.3s), bar fills (1s), warp-speed zoom+blur (1.5s), dashboard fades in.
- **Skill bars:** Width animates from 0 to target with 200ms stagger per bar, 2s duration, cubic-bezier easing.
- **Skeleton loaders:** Opacity pulses 0.4 to 0.7 at 1.5s intervals (shimmerPulse).
- **Click particles:** 10-15 particles burst from clicked element perimeter, slow floaty physics (gravity 0.002), 4 brand colors.
- All animations respect prefers-reduced-motion media query.

2.8 Responsive Strategy

Mobile-first is not used. The app is desktop-first with mobile breakpoints.

Breakpoint	Changes
< 768px (mobile)	Sidebar becomes bottom tab bar (64px). Grids collapse to 1 column. Studio split view stacks vertically. Phase pills hide text
< 900px	Stats grid collapses to single column.
< 480px	Studio welcome input stacks vertically. Quiz cards lose aspect ratio constraint.

Sidebar-to-tab-bar pattern: The 240px fixed left sidebar transforms into a 64px fixed bottom tab bar on mobile. Nav items switch from icon+label rows to icon+label columns. The active indicator moves from a left border to a top border.

2.9 Icon System

All icons are inline SVGs in the Lucide style: 20-24px, 1.5px stroke, round caps and joins,currentColor for stroke (inherits text color). No icon library is imported -- each component declares its own SVG icon functions. This means icons are duplicated across files, but it avoids external dependencies.

Recommendation for rebuild: Use a proper icon library (Lucide React, Heroicons, etc.) instead of inline SVGs to eliminate duplication.

3. Technical Architecture

This section describes the current implementation decisions in a way that's useful regardless of what framework or language you choose for the rebuild. Focus on the "what" and "why" rather than the specific React/FastAPI code.

3.1 System Overview

The current stack is React (Create React App) for the frontend and FastAPI (Python) for the backend, with SQLite as the database. The backend serves the built frontend as static files in production and handles OAuth, API routes, and AI integration.

Layer	Current Tech	Notes
Frontend	React 19, CRA, plain CSS	No CSS modules, no Tailwind, no TypeScript
Backend	FastAPI 0.128, Uvicorn	Python 3.10+, async
Database	SQLite via SQLAlchemy ORM	Single file DB, auto-created on startup
Auth	GitHub OAuth via Authlib	Session cookies, no JWT
AI	OpenRouter API (OpenAI client)	DeepSeek models, credit-budgeted
Deployment	Docker (multi-stage build)	Node for frontend build, Python for runtime

Create React App (CRA) -- A tool that sets up a React project with webpack bundling, development server, and build scripts. Now considered legacy -- alternatives include Vite, Next.js, and Remix.

FastAPI -- A modern Python web framework for building APIs. Async-capable, auto-generates API documentation, uses Python type hints for validation.

3.2 Authentication Flow

Step-by-step flow:

- 1. User clicks "Sign in with GitHub" on the login page.
- 2. Frontend redirects to backend: GET /auth/login
- 3. Backend redirects to GitHub's OAuth authorization page (scopes: user:email, read:user).
- 4. User approves. GitHub redirects back to GET /auth/callback with a code.
- 5. Backend exchanges code for access token, calls GitHub API for user profile.
- 6. Backend creates/updates user in DB, stores github_id and username in session cookie.
- 7. Backend redirects to frontend /dashboard.
- 8. Frontend checks GET /auth/me on every protected page load to verify session.

Key architecture decisions: Session cookies (not JWT) keep auth simple -- the browser automatically sends the cookie on every request, no token management needed on the frontend. The session is signed with a secret key to prevent tampering.

3.3 API Endpoints

Authentication

Method	Path	Auth?	Purpose
GET	/auth/login	No	Start GitHub OAuth flow (redirect)
GET	/auth/callback	No	OAuth callback -- creates/updates user
GET	/auth/logout	No	Clear session, redirect to login
GET	/auth/me	Yes	Returns {id, github_id, username, role}

Quiz System

Method	Path	Auth?	Purpose
GET	/api/quiz/questions	No	All quizzes with questions
GET	/api/quiz/quiz/{quiz_id}	No	Single quiz by ID
POST	/api/quiz/submit	Yes	Submit answers. Body: {quiz_id, answers: [{question_id, answer}]}

Leaderboard

Method	Path	Auth?	Purpose
GET	/api/leaderboard/	No	Top 10 users by total points
GET	/api/leaderboard/user/{username}	No	All scores for a specific user

Courses & Quests

Method	Path	Auth?	Purpose
GET	/api/courses/{slug}/quests	No	List all quests for a course
GET	/api/courses/{slug}/quests/{id}	No	Single quest definition
GET	/api/courses/{slug}/progress	Yes	Student progress: {xp, streak, completed_quest_ids}
POST	/api/courses/{slug}/quests/{id}/complete	Yes	Record completion, returns {xp, streak_count}

AI Tutor

Method	Path	Auth?	Purpose
POST	/api/ai/tutor-chat	Yes	Send message to AI tutor. Body: {quest_id, message, history}
GET	/api/ai/credits?quest_id=...	Yes	Check remaining credits: {budget, remaining}
POST	/api/ai/grade-response	Yes	Submit conversation for rubric grading

3.4 Data Models

Five database tables. The current implementation uses SQLite with SQLAlchemy ORM, but any relational database would work.

Table	Key Columns	Purpose
users	id, github_id (unique), username, role	User accounts (created on first OAuth login)
scores	user_id (FK), quiz_id, score, timestamp, correct_questions (JSON)	Question attempts
course_progress	user_id (FK), course_slug, xp, streak_count, last_activity_at	Per-course progress tracking
quest_completions	user_id (FK), quest_id, completed_at, artifact_ref	Quest completion records
credit_ledger	user_id (FK), quest_id, model_tier, tokens_spent	AI budget usage log

Design note: The scores table stores a correct_questions JSON column that tracks which questions were answered correctly on each attempt. This enables the retake scoring logic (only newly-correct answers earn points).

3.5 AI Integration Details

- **Provider:** OpenRouter (openrouter.ai) -- an API aggregator compatible with the OpenAI client library.
- **Fast model:** deepseek/deepseek-v4-flash -- used for tutor chat and rubric grading.
- **Agent model:** deepseek/deepseek-v4-pro -- reserved for future features (e.g., generating bugs).
- **Credit system:** Each quest defines a credit_budget in its YAML file. Usage is tracked in the credit_ledger table as tokens_spent. Remaining = budget - SUM(tokens_spent). HTTP 402 when exhausted.
- **Rate limiting:** In-memory sliding window -- 6 calls per 60 seconds per user. HTTP 429 when exceeded.
- **System prompts:** Loaded from markdown files at backend/prompts/. One for tutoring (Socratic style), one for rubric grading.

***Sliding window rate limiter** -- Tracks the timestamps of recent API calls. If more than N calls happened in the last M seconds, the next call is rejected. "Sliding" means old calls drop off as time passes (unlike a fixed-window approach).*

3.6 Content System

Quiz and course content is defined in YAML files, not in the database. This makes content easy to version-control, review in pull requests, and edit without database access.

- **Quiz files:** backend/quizzes/{category}/{quiz_name}.yaml -- 12 files across 3 categories (Python, Java, Technology). Each defines id, title, description, category, and a list of questions.
- **Course files:** backend/courses/{course-slug}/ -- one course (build-real-stuff) with 2 quest YAML files. Each quest defines id, type, stage, credit_budget, prompt, and optional rubric.
- **AI prompts:** backend/prompts/*.md -- system prompts loaded at startup.

***YAML** -- A human-readable data format (like JSON but cleaner). Used here instead of a database for content because it's easy to edit in a text editor and track in Git.*

3.7 Performance Patterns

- **Auth prefetching:** On successful login, the app immediately fetches dashboard data (leaderboard, quizzes, scores, course progress) in parallel. Consumed once via `getPrefetchedData()` with a 30-second TTL.
- **API response caching:** The frontend API wrapper caches GET responses with per-endpoint TTLs (5 seconds for credits, 30 seconds for leaderboard, 5 minutes for quizzes). Mutations auto-invalidate related caches.
- **Lazy loading:** All page components use `React.lazy()` + `Suspense` for code splitting.
- **Skeleton UI:** Loading states show animated skeleton placeholders instead of blank screens or spinners.
- **Canvas animation:** Frame-rate capped at 30fps. Animations only run when particles/elements exist.

***Code splitting** -- Loading page JavaScript only when that page is visited, rather than bundling everything into one large file. Reduces initial load time.*

***Skeleton UI** -- Gray placeholder shapes that mimic the layout of real content while data loads. Feels faster than a spinner because the user sees the page structure immediately.*

3.8 Environment & Configuration

Variable	Default	Purpose
GITHUB_CLIENT_ID	(required)	GitHub OAuth app client ID
GITHUB_CLIENT_SECRET	(required)	GitHub OAuth app client secret
GITHUB_REDIRECT_URI	localhost:8000/auth/callback	OAuth callback URL
SECRET_KEY	(required)	Session cookie signing key
DATABASE_PATH	./quiz_app.db	Path to SQLite database file
FRONTEND_URL	(empty)	Frontend URL for CORS and redirects
OPENROUTER_API_KEY	(empty)	API key for AI features
OPENROUTER_MODEL_FAST	deepseek/deepseek-v4-flash	Model for chat/grading
OPENROUTER_MODEL_AGENT	deepseek/deepseek-v4-pro	Model for future features
AI_RATE_LIMIT_MAX_CALLS	6	Max AI calls per user per window
AI_RATE_LIMIT_WINDOW_SECONDS	60	Rate limit window duration
REACT_APP_API_URL	localhost:8000	Frontend -> backend API base URL

4. Jargon Glossary

Quick reference for all technical terms used in this document.

API (Application Programming Interface)

A set of rules for how software components communicate. In this app, the frontend sends HTTP requests to the backend API to get data and perform actions.

CORS (Cross-Origin Resource Sharing)

A browser security feature that controls which websites can make requests to your backend. Required when frontend and backend run on different ports/domains.

CRUD

Create, Read, Update, Delete -- the four basic database operations.

CSS Custom Properties

Variables defined in CSS (e.g., `--color-primary: #ef6c00`) that can be reused throughout stylesheets. Change the value once, update everywhere.

Design Tokens

The smallest pieces of a design system: colors, spacing, fonts, radii, shadows. Named abstractly for reuse.

Endpoint

A specific URL path that the backend responds to (e.g., `/api/quiz/submit`). Each endpoint handles a specific action.

FK (Foreign Key)

A database column that references another table's primary key, creating a relationship between tables.

Gamification

Using game elements (points, levels, streaks, leaderboards) in a non-game context to drive engagement.

JWT (JSON Web Token)

A self-contained token for auth. NOT used in this app (we use session cookies instead), but mentioned for comparison.

Lazy Loading

Loading resources (code, images) only when needed instead of all at once. Improves initial load time.

OAuth

A protocol letting users log in via third-party services (GitHub, Google) without sharing passwords.

OpenRouter

An API aggregator that provides access to many AI models through a single API endpoint.

ORM (Object-Relational Mapping)

A library (like SQLAlchemy) that lets you interact with a database using programming language objects instead of raw SQL.

Parsons Problem

A code ordering exercise where students drag scrambled lines into the correct sequence.

Prefetching

Loading data before it's needed so the UI appears instantly when the user navigates to a page.

SDLC (Software Development Lifecycle)

The phases of building software: Discover, Analyze, Design, Build, Ship.

Session Cookie

A browser-stored token identifying a logged-in user. Signed to prevent tampering.

Skeleton UI

Placeholder shapes mimicking content layout while data loads. Feels faster than a spinner.

Sliding Window Rate Limiter

Tracks recent API calls in a time window. Rejects calls exceeding the limit. Old calls drop off as time passes.

Token (AI)

A unit of text for AI models. Roughly 4 characters = 1 token. Used to measure and budget AI usage.

YAML

A human-readable data format used for configuration and content files. Cleaner syntax than JSON.